02393 Programming in C++

Module 8: Inheritance

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Inheritance
- Derived classes
- Abstract classes
- Lab

A recap from the previous lectures

- The structure of a C++ program
  - #include and #define directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
  - cin, cout
- Variables, values and types
  - string, int, double, float, bool, arrays (statically and dynamically allocated), pointers, enum, struct, vector, ifstream, ofstream, class, this
- Expressions
  - some numeric and boolean operators and math functions, conditional expressions
- Statements
  - if, while, for, switch

Recap on Object-Oriented Programming

- Object-Oriented programming focuses on data:
  - Data types and operations to manipulate them are grouped in classes
  - Classes are the main elements of a program
  - To represent data, classes are instantiated into objects
- A class is similar to a struct, but its members can be:
  - Variables
  - Methods (a bit like functions)
- Class members can be public or private
  - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
  - Constructor: called when an object is created, either statically, or dynamically using new
  - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete
  - Assignment: customizes the behavior of the = operator.

Recap on Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
  - Specify allowed operations on an ADT, by making them public
  - Hide everything else, by making it private
  - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed
- We can use templates to make our code generic and reusable (e.g., for containers)

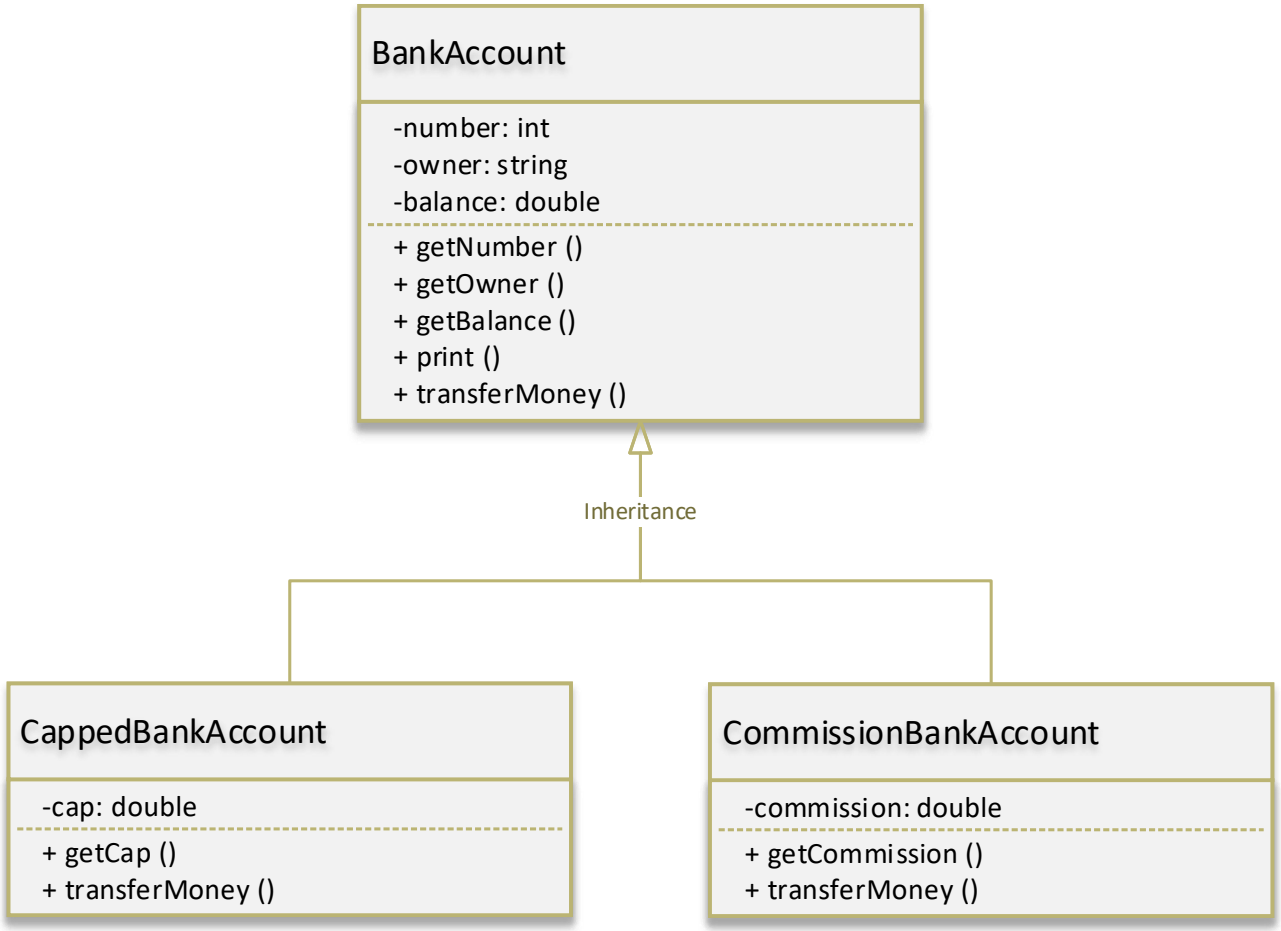
Extending BankAccount

- During lecture 6, we implemented a class BankAccount to handle bank accounts
- We need to extend BankAccount to support two new kinds of bank account:
  - CappedBankAccount
    - Transfers have a cap
      - When money is transferred, if the amount being transferred exceeds the cap, the transfer fails
  - CommissionBankAccount
    - Transfers are subject to account-specific commission
      - When money is transferred, the sender is charged the commission

Inheritance

- A subtyping relation says: every instance of a subtype is (also) an instance of a supertype
  - In arithmetic, every integer number is a real number
  - In geometry, every square is a rectangle
  - In a program for managing bank accounts, every CappedBankAccount is a BankAccount
- The supertype supports general operations; the subtype may have specialized operations
- When is this useful?
  - Bottom-up perspective (generalization)
    - We have many bank account classes with shared functionalities. let's group them together
  - Top-down perspective (specialization)
    - A BankAccount class distinguishes different kinds of bank accounts. let's make separate classes
- Advantages: modularity, clarity, maintainability

Class diagrams



Derived classes

```
1  class BaseClass {
2      private:
3          int var;
4          void increment(int);
5      public:
6          BaseClass(int);
7          int getVar();
8          void print();
9  }
10
11 class DerivedClass : public BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
```

- When defining a class, we can specify that the class is derived from another class
  - Public members are inherited
  - Private methods are not
  - Private attributes are inherited, but not accessible by the derived class

Derived classes

```
1  class BaseClass {
2      private:
3          int var;
4          void increment(int);
5      public:
6          BaseClass(int);
7          int getVar();
8          void print();
9  }
10
11 class DerivedClass : public BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
```

- When defining a class, we can specify that the class is derived from another class
  - Public members are inherited
  - Private methods are not
  - Private attributes are inherited, but not accessible by the derived class
- The derived class can have its own members, in addition to the inherited ones

Derived classes

```
1  BaseClass::BaseClass(int i) {
2      this->var = i;
3  }
4
5  void BaseClass::print(){
6      cout << "the value of var is " << this->var << endl;
7  }
8
9  DerivedClass::DerivedClass(string s) : BaseClass(10) {
10     this->text = s;
11 }
12
13 void DerivedClass::print(){
14     cout << "the value of text is " << this->text << ", and ";
15     BaseClass::print();
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
  - Private attributes of the base class cannot be accessed by the derived class
  - How to initialize them?

Derived classes

```
1  BaseClass::BaseClass(int i) {
2      this->var = i;
3  }
4
5  void BaseClass::print(){
6      cout << "the value of var is " << this->var << endl;
7  }
8
9  DerivedClass::DerivedClass(string s) : BaseClass(10) {
10     this->text = s;
11 }
12
13 void DerivedClass::print(){
14     cout << "the value of text is " << this->text << ", and ";
15     BaseClass::print();
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
  - Private attributes of the base class cannot be accessed by the derived class
  - How to initialize them?
  - By calling the constructor of the base class

Derived classes

```
1  BaseClass::BaseClass(int i) {
2      this->var = i;
3  }
4
5  void BaseClass::print(){
6      cout << "the value of var is " << this->var << endl;
7  }
8
9  DerivedClass::DerivedClass(string s) : BaseClass(10) {
10     this->text = s;
11 }
12
13 void DerivedClass::print(){
14     cout << "the value of text is " << this->text << ", and ";
15     BaseClass::print();
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
  - Private attributes of the base class cannot be accessed by the derived class
  - How to initialize them?
  - By calling the constructor of the base class
- We can override (i.e., redefine) inherited methods, so to specialize their code
  - The overridden method must be specified in the derived class definition

Derived classes

```
1  BaseClass::BaseClass(int i) {
2      this->var = i;
3  }
4
5  void BaseClass::print(){
6      cout << "the value of var is " << this->var << endl;
7  }
8
9  DerivedClass::DerivedClass(string s) : BaseClass(10) {
10     this->text = s;
11 }
12
13 void DerivedClass::print(){
14     cout << "the value of text is " << this->text << ", and ";
15     BaseClass::print();
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
  - Private attributes of the base class cannot be accessed by the derived class
  - How to initialize them?
  - By calling the constructor of the base class
- We can override (i.e., redefine) inherited methods, so to specialize their code
  - The overridden method must be specified in the derived class definition
  - We can reuse the method in the base class by invoking it in the overridden method

Encapsulation and inheritance

```
1  class BaseClass {
2      public:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 void DerivedClass::print(){
13     cout << "the contents are the following: ";
14     cout << "text is " << this->text <<
15     cout << ", and var is " << this->var << endl;
16 }
```

- Public members are accessible from outside the class
  - They break the encapsulation principle
- Private members are not accessible from derived classes
  - We rely on public methods of the base class to interact with them
  - In many cases, this is not enough
- Solution: declare members that must be accessible from derived class as protected:
  - Inherited by derived classes
  - Accessible by derived classes
  - Non accessible from outside the class

Encapsulation and inheritance

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : public BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
```

- What happens to the interface of a derived class?

Encapsulation and inheritance

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : public BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
  - With the public modifier:
    - Public members will remain public
    - Protected members will remain protected

Encapsulation and inheritance

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : protected BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
  - With the public modifier:
    - Public members will remain public
    - Protected members will remain protected
  - With the protected modifier:
    - Public members will become protected
    - Protected members will remain protected

Encapsulation and inheritance

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
  - With the public modifier:
    - Public members will remain public
    - Protected members will remain protected
  - With the protected modifier:
    - Public members will become protected
    - Protected members will remain protected
  - With the private modifier:
    - Public members will become private
    - Protected members will become private

Encapsulation and inheritance

Derived class	Public members	Protected members	Private members
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Private	Private	Private

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?
  – DerivedClass::print()

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?
  – DerivedClass::print()
- Which method is invoked by b-> print()?

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?
  - DerivedClass::print()
- Which method is invoked by b-> print()?
  - BaseClass::print()
  - Why?
  - Because C++ uses static method dispatch:
    - The method is determined by the data type, and not by the object

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?
  - DerivedClass::print()
- Which method is invoked by b-> print()?
  - BaseClass::print()
  - Why?
  - Because C++ uses static method dispatch:
    - The method is determined by the data type, and not by the object
- Can we force DerivedClass::print() to be invoked instead?

Overridden methods dispatch

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          int getVar();
9          virtual void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by d-> print()?
  - DerivedClass::print()
- Which method is invoked by b-> print()?
  - BaseClass::print()
  - Why?
  - Because C++ uses static method dispatch:
    - The method is determined by the data type, and not by the object
- Can we force DerivedClass::print() to be invoked instead?
  - Yes, with dynamic method dispatch
    - We have to mark print() as virtual in BaseClass
  - Slower but usually more intuitive

Abstract classes

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          virtual void print() = 0;
9  }
10
11 class DerivedClass : private BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
19
20 void DerivedClass::print(){
21     cout << "the contents are the following: ";
22     cout << "text is " << this->text <<
23     cout << ", and var is " << this->var << endl;
24 }
```

- A class is abstract if it contains at least one “pure virtual” method:
  - Marked by “= 0”
  - Defined but not implemented

Abstract classes

```
1  class BaseClass {
2      private:
3          void increment(int);
4      protected:
5          int var;
6      public:
7          BaseClass(int);
8          virtual void print() = 0;
9  }
10
11 class DerivedClass : private BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
19
20 void DerivedClass::print(){
21     cout << "the contents are the following: ";
22     cout << "text is " << this->text <<
23     cout << ", and var is " << this->var << endl;
24 }
```

- A class is abstract if it contains at least one “pure virtual” method:
  - Marked by “= 0”
  - Defined but not implemented
- Abstract classes cannot be instantiated, only derived
- A class derived from an abstract class can be instantiated only if it implements all the pure virtual methods of the base class

Lab time

- Today
  - Make sure C++ works on your computer, request help if it doesn’t
  - Begin working on Assignment 8
  - Ask questions if something is unclear (including previous assignments)


DTU